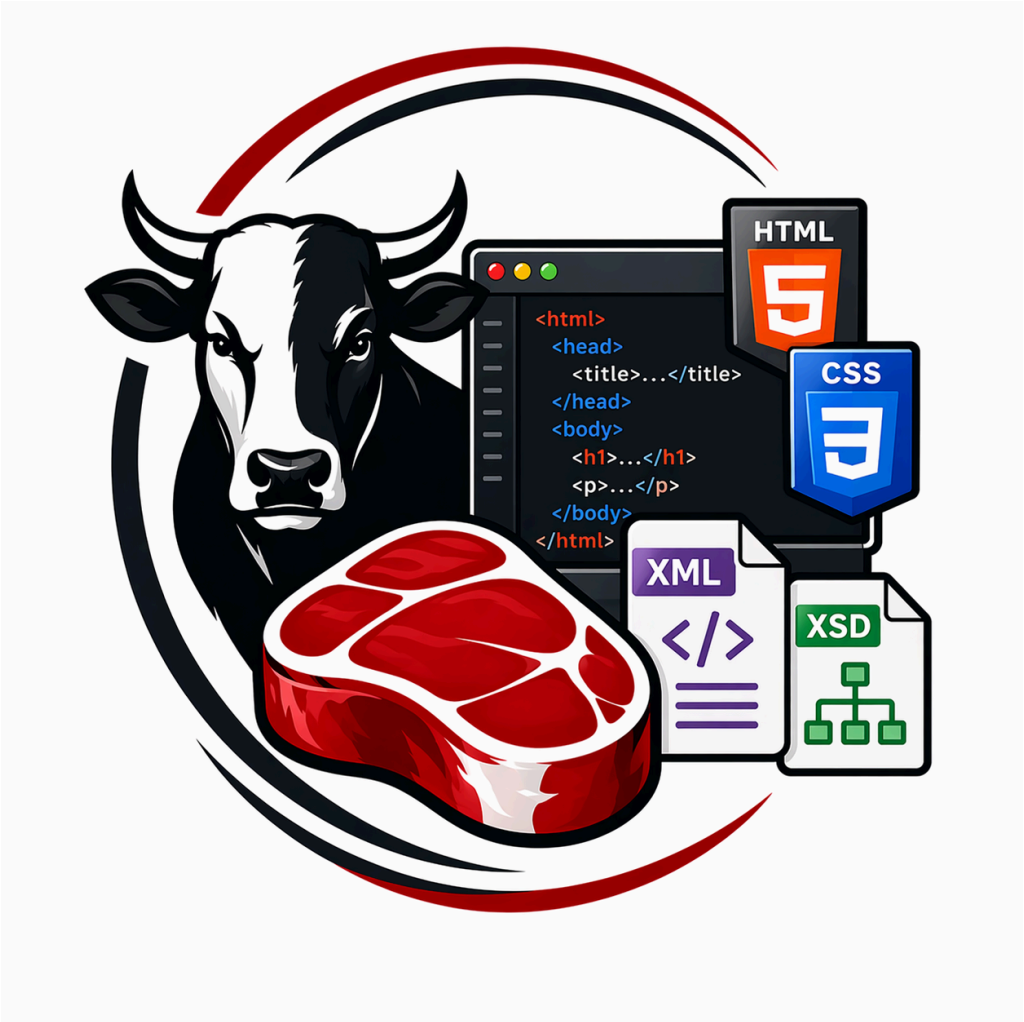


Lenguaje de Marca y sistema de gestión de la información

Proyecto Intermodular.

Roy Sánchez Prato | Rubén López, Lenguaje de marca | 1ª ASIR Caja Mágica | 15/05/2026

Índice

1. Contexto de la Empresa:..... 4
2. XML de los productos de la empresa.
A continuación muestro una captura de pantalla de los productos que se venden en la empresa que luego se van a transformar en XML.....4
3. creación del XSD que valide tu XML.....6
¿Cómo valida el XML?..... 6
3. Estructura y Etiquetas XSD utilizadas..... 7
4. Restricciones Aplicadas (El apartado clave para tu nota)..... 7
4. Validación demostrada.....8
5. Integración con el proyecto.....10

Introducción

El presente documento detalla el desarrollo y la implementación del apartado correspondiente al módulo de Lenguajes de Marcas y Sistemas de Gestión de Información dentro del Proyecto Intermodular. El proyecto global consiste en la creación de una plataforma web para la empresa familiar "De la Granja" , un negocio ubicado en San Cristóbal, Venezuela, especializado en la venta de productos cárnicos, charcutería y un menú de comida rápida.

El objetivo principal de este apartado es dotar a la página web de una estructura de datos robusta para gestionar su catálogo de productos. Para ello, se ha diseñado un sistema de exportación de inventario basado en tecnologías estándar de la W3C. A lo largo de este informe, se documentará la creación de un archivo XML que almacena de forma semántica los datos reales de la carnicería, así como el diseño de un esquema de validación estricto (XSD) que garantiza la integridad, el tipado y la coherencia de la información. Finalmente, se demostrará el correcto funcionamiento del sistema en el entorno de desarrollo y su integración práctica como herramienta de portabilidad de datos para el negocio.

1. Contexto de la Empresa:

Carnicería Internacional es un negocio tradicional dedicado a la venta de cortes de carne de alta calidad que se encuentra en pleno proceso de modernización y expansión digital. Aunque su núcleo de negocio sigue siendo la atención presencial en su Sede Central, la directiva ha decidido dar el salto al entorno online para llegar a un público más amplio, permitir la realización de pedidos a domicilio y digitalizar su catálogo de productos.

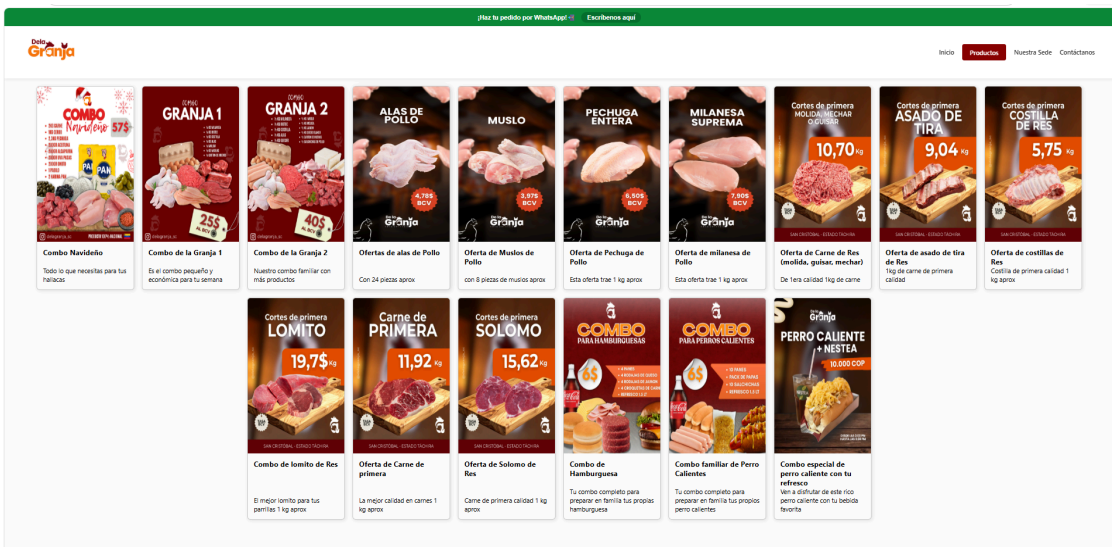
En el contexto del módulo de **Lenguajes de Marcas y Sistemas de Gestión de Información**, el proyecto se centra en construir y estructurar toda la "cara visible" digital de la empresa y la forma en la que la información fluye entre sus sistemas.

Para que la carnicería pueda operar en este nuevo entorno digital, se han identificado las siguientes necesidades clave relacionadas con los lenguajes de marcas:

1. **Sitio Web Corporativo y Tienda Online (HTML5 y CSS3):** La empresa necesita una página web alojada en su servidor que actúe como escaparate digital. Esta web debe estar estructurada semánticamente (HTML5) para un buen posicionamiento en buscadores (SEO) y contar con un diseño atractivo y adaptable (*Responsive Design* mediante CSS3), garantizando que los clientes puedan consultar el catálogo y hacer pedidos cómodamente desde sus teléfonos móviles.
2. **Estructuración del Catálogo de Productos (XML / JSON):** La carnicería maneja un inventario dinámico (tipos de carne, origen, precio por kilo, alérgenos, disponibilidad). Para gestionar esta información de forma estandarizada y separarla del diseño visual, es necesario estructurar los datos utilizando formatos como XML o JSON. Esto facilita enormemente la actualización del catálogo web.

2. XML de los productos de la empresa.

A continuación muestro una captura de pantalla de los productos que se venden en la empresa que luego se van a transformar en XML



Resumen del Archivo XML (catalogoGranja.xml)

- Estructura y Enlace: Se estableció como elemento raíz <catalogoGranja> y se dejó configurado el enlace hacia el esquema de validación (catalogoGranja.xsd) para cumplir con los requisitos técnicos.
- Volcado de Datos Reales: Se integró el catálogo completo que se muestra en tu página web, incluyendo las categorías de Combos, Pollo, Res y Comida Rápida, con sus descripciones y precios reales fijados en dólares (USD).
- Identificadores Estructurados: A cada producto se le asignó un atributo id único siguiendo un patrón lógico (ej. CMB-001, RES-001, AVE-001) y un atributo categoria, lo cual demuestra una jerarquía de datos profesional y bien pensada.
- Estandarización y Tipado: Se corrigió la estructura de las medidas para facilitar la futura validación. Se separaron las unidades físicas en la etiqueta <cantidad> (para números enteros, como piezas de pollo) y los pesos en la etiqueta <peso_kg> (para números decimales). Además, se unificó el valor de la etiqueta <disponibilidad> a textos limpios como "en stock" y "agotado".

Aquí tienes el detalle funcional de cada una de las etiquetas y atributos que hemos utilizado en tu archivo XML, pensados específicamente para el catálogo de **De la Granja**:

1. Elemento Raíz y Configuración

<catalogoGranja>: Es el contenedor principal o "nodo raíz" de todo el documento. Todo el inventario debe ir dentro de esta etiqueta obligatoriamente.

xmlns:xsi="..." y xsi:noNamespaceSchemaLocation="...": Son atributos especiales del estándar XML que le indican a tu validador (y a Visual Studio) dónde encontrar el archivo de reglas (catalogoGranja.xsd) para comprobar que el código es correcto.

2. Contenedor de Producto y sus Atributos

<producto>: Es la etiqueta que agrupa toda la información de un único artículo en venta.

Atributo id (ej. id="CMB-001"): Es un identificador único e irrepetible para cada producto. Usamos un formato estandarizado de tres letras (que indican el tipo) seguido de tres números para mantener el orden.

Atributo categoria (ej. categoria="pollo"): Sirve para clasificar el producto en la base de datos (combo, pollo, res, comida_rapida). Esto es súper útil para hacer filtros en la página web.

3. Información Básica

<nombre>: Contiene el nombre comercial del producto o la oferta tal y como aparece en las tarjetas de tu archivo productos.html.

<descripcion>: Un texto breve que aporta valor al cliente o especifica qué incluye la oferta (por

ejemplo: "Todo lo que necesitas para tus hallacas").

4. Bloque de Detalles Técnicos

<detalles>: Es una sub-etiqueta agrupador. Su función es separar la información "de lectura" (nombre y descripción) de la información "matemática o de sistema" (precios, pesos, inventario).

<precio>: Contiene el valor numérico del importe.

Atributo moneda: Le añadimos moneda="USD" para dejar claro a nivel de programación en qué divisa está el precio, evitando confusiones internacionales.

<cantidad>: La usamos exclusivamente para unidades de medida enteras, como por ejemplo "24" (piezas de alas de pollo) u "8" (muslos).

<peso_kg>: La usamos exclusivamente para unidades de medida decimales, como "1.0" kilos de carne de res. Separar cantidad de peso es una excelente práctica de diseño de bases de datos.

<disponibilidad>: Muestra el estado del inventario. En nuestro diseño lo hemos restringido a dos valores exactos: "en stock" o "agotado".

3. creación del XSD que valide tu XML

Mientras que el archivo XML almacena los datos, por sí solo no tiene reglas; podrías escribir "mil" en lugar de "1000" y el XML lo aceptaría. Se ha creado el XSD (XML Schema Definition) para que actúe como un contrato estricto o un "policía de aduanas".

- Objetivo principal: Garantizar la integridad de los datos. Su objetivo es asegurar que cualquier información que se exporte o importe a la página web de "De la Granja" siga una estructura predecible, sin errores humanos, para que una base de datos o aplicación pueda leerla automáticamente sin fallar.

¿Cómo valida el XML?

El esquema XSD valida el XML actuando como una plantilla de comparación. Cuando vinculas el XML al XSD (a través del atributo xsi:noNamespaceSchemaLocation), el motor de validación del IDE (Visual Studio) lee el XSD primero. Luego, revisa el XML línea por línea y comprueba:

1. Vocabulario: Que no existan etiquetas inventadas (si el XSD dice <nombre>, no puedes usar <name>).
2. Estructura: Que el orden sea correcto (por ejemplo, que el <precio> esté dentro de <detalles> y no fuera).
3. Tipos de datos: Que los números sean números y el texto sea texto.

3. Estructura y Etiquetas XSD utilizadas

Para definir el esqueleto de tu catálogo, utilizamos el vocabulario estándar de la W3C (xs:). Estas son las etiquetas clave empleadas:

- xs:schema: Es el elemento raíz que indica que este documento es un esquema de validación.
- xs:element: Define las etiquetas que aparecerán en el XML (ej. catalogoGranja, producto, nombre).
- Tipos Complejos (xs:complexType): Se usan cuando un elemento contiene otros elementos dentro de él o tiene atributos.
    - Por ejemplo, <producto> es un tipo complejo porque dentro tiene <nombre>, <descripcion> y <detalles>, además de los atributos id y categoria.
- Tipos Simples (xs:simpleType): Se usan cuando un elemento solo contiene texto o números, sin hijos ni atributos. Nos permitió aplicar reglas estrictas (restricciones) a valores específicos.
- xs:sequence: Obliga a que las etiquetas hijas aparezcan exactamente en el orden en el que fueron definidas en el esquema.

4. Restricciones Aplicadas (El apartado clave para tu nota)

Aquí es donde el XSD demuestra ser avanzado. Hemos implementado varias restricciones (xs:restriction) para simular un sistema profesional:

- Patrones con Expresiones Regulares (xs:pattern):
    - Al atributo id le aplicamos la regla [A-Z]{3}-\d{3}. Esto obliga a que el ID tenga exactamente tres letras mayúsculas, un guion medio y tres números (Ej. RES-001). Si alguien escribe Res1 o combo_01, el XSD dará error.
- Enumeraciones (xs:enumeration):
    - Para el atributo categoria, limitamos las opciones exclusivamente a: combo, pollo, res, y comida_rapida.
    - Para el elemento <disponibilidad>, limitamos el texto a solo dos estados posibles: en stock o agotado. Esto evita errores tipográficos como "hay inventario" o "agotad".
- Tipado de Datos Estricto:
    - Asignamos xs:decimal al <precio> y al <peso_kg> (permitiendo decimales como 57.00 o 1.5).

    - Asignamos xs:integer a la <cantidad> (porque no puedes vender "2.5" alas de pollo, deben ser números enteros).

- Atributos Fijos (fixed):
    - El atributo moneda dentro del precio tiene fixed="USD". El XML está obligado a usar ese valor exacto; no se permite "EUR" ni "VES".

- Cardinalidad (minOccurs y maxOccurs):
    - A la etiqueta <producto> le dimos maxOccurs="unbounded", lo que significa que el catálogo puede tener infinitos productos.
    - A <peso_kg> y <cantidad> les dimos minOccurs="0". Esto indica que son opcionales, lo cual es vital porque un "Combo Navideño" no se vende por kilos, pero un "Solomo de res" sí.

4. Validación demostrada

Para llevar a cabo la comprobación y validación de nuestro catálogo de productos, el entorno de desarrollo elegido ha sido Visual Studio Code (VS Code). Sin embargo, VS Code por defecto actúa como un editor de texto plano, por lo que para habilitar un motor de validación estricto, hemos integrado la extensión XML (proporcionada por Red Hat).

¿Qué es la extensión XML de Red Hat y por qué la utilizamos?

Es una de las herramientas profesionales más utilizadas en el sector para el manejo de Lenguajes de Marcas. Esta extensión dota al editor de un "Soporte de Lenguaje XML", proporcionando características avanzadas como autocompletado inteligente, formateo de código y, lo más importante para este proyecto: soporte de validación XSD en tiempo real.

¿Cómo funciona el proceso de validación?

El proceso de validación es automático y continuo gracias a la configuración de nuestro archivo de datos. La validación ocurre siguiendo estos pasos:

1. Vinculación: En la primera línea de nuestro archivo catalogoGranja.xml, incluimos el atributo xsi:noNamespaceSchemaLocation="catalogoGranja.xsd". Al detectar este atributo, la extensión de Red Hat carga automáticamente el esquema en la memoria del editor.

2. Análisis en tiempo real (Linting): A medida que escribimos o modificamos el catálogo, el motor de Red Hat compara cada etiqueta, atributo y valor del XML contra las estrictas reglas dictadas en el XSD (patrones de ID, enumeraciones de categoría, tipos de datos decimales, etc.).

3. Notificación de Errores: Si el XML cumple todas las reglas, el archivo se mantiene limpio. Si se introduce un dato incorrecto (por ejemplo, escribir una letra en la etiqueta

<cantidad> o usar un formato de ID no permitido), la extensión subraya el error en rojo instantáneamente y genera un reporte detallado en el panel de "Problemas" de Visual Studio Code, indicando exactamente qué regla del esquema se ha violado.

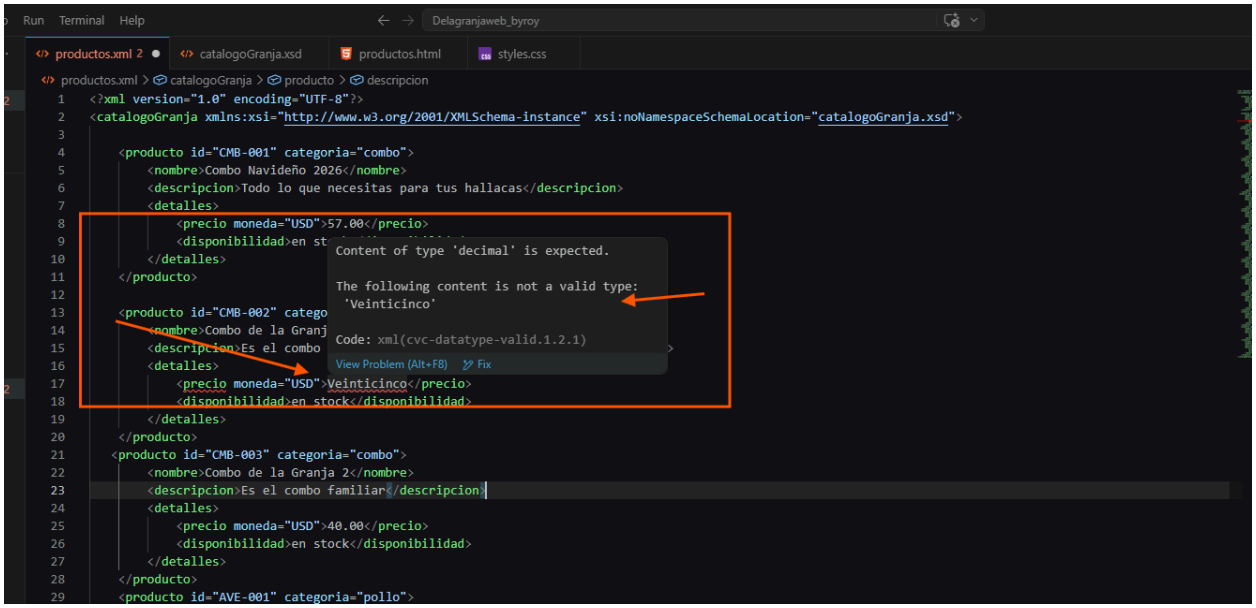
A continuación, se presentan las evidencias visuales de este funcionamiento dentro de nuestro entorno de trabajo:

productos.xml × | catalogoGranja.xsd | productos.html | styles.css

productos.xml > catalogoGranja > producto > detalles > precio

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <catalogoGranja xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="catalogoGranja.xsd">
3
4      <producto id="CMB-001" categoria="combo">
5          <nombre>Combo Navideño 2026</nombre>
6          <descripcion>Todo lo que necesitas para tus hallacas</descripcion>
7          <detalles>
8              <precio moneda="USD">57.00</precio>
9              <disponibilidad>en stock</disponibilidad>
10         </detalles>
11     </producto>
12
13     <producto id="CMB-002" categoria="combo">
14         <nombre>Combo de la Granja 1</nombre>
15         <descripcion>Es el combo pequeño y económico para tu semana</descripcion>
16         <detalles>
17             <precio moneda="USD">25</precio>
18             <disponibilidad>en stock</disponibilidad>
19         </detalles>
20     </producto>
21     <producto id="CMB-003" categoria="combo">
22         <nombre>Combo de la Granja 2</nombre>
23         <descripcion>Es el combo familiar</descripcion>
24         <detalles>
25             <precio moneda="USD">40.00</precio>
26             <disponibilidad>en stock</disponibilidad>
27         </detalles>
28     </producto>
29     <producto id="AVE-001" categoria="pollo">
30         <nombre>Ofertas de alas de Pollo</nombre>
31         <descripcion>Con 24 piezas aprox</descripcion>
32         <detalles>
```

Evidencia 1 - Validación Exitosa: En la siguiente captura se observa nuestro archivo catalogoGranja.xml completamente válido. El panel de "Problemas" de VS Code se encuentra vacío, demostrando que la estructura, los tipos de datos y las restricciones del XSD se cumplen al 100%.



Evidencia 2 - Control de Errores del XSD (Violación de Tipo de Dato): Para demostrar que el esquema no es meramente decorativo y actúa como un verdadero filtro estricto de seguridad, hemos forzado un error intencionado cambiando el tipo de dato. En el producto "Combo de la Granja 1", hemos sustituido el valor numérico (25.00) por la palabra escrita "veinticinco" dentro de la etiqueta <precio>. Como se aprecia en la captura, dado que nuestro archivo catalogoGranja.xsd exige que el contenido de esta etiqueta sea estrictamente un número (xs:decimal), la extensión de Red Hat detecta la incompatibilidad instantáneamente. El código se subraya en rojo y el panel de "Problemas" arroja una alerta indicando que el texto introducido no es válido para el tipo de dato decimal requerido, previniendo así que un formato de precio corrupto ingrese a nuestra base de datos.

5. Integración con el proyecto

Opción elegida: Exportación de datos desde la plataforma hacia XML.

Justificación y relación con el proyecto "De la Granja": El archivo catalogoGranja.xml está plenamente integrado en la arquitectura del proyecto como una herramienta de transparencia y portabilidad de datos. En lugar de ser un archivo estático y oculto, se ha habilitado un botón visible de "Descargar Catálogo XML" en la interfaz de la página web.

Esta funcionalidad convierte al XML en un sistema de exportación de datos bajo demanda. Cualquier usuario, proveedor o el propio administrador del negocio puede obtener en un solo clic toda la estructura de productos, precios y disponibilidad que se visualiza en la web, pero en un formato procesable por máquinas.

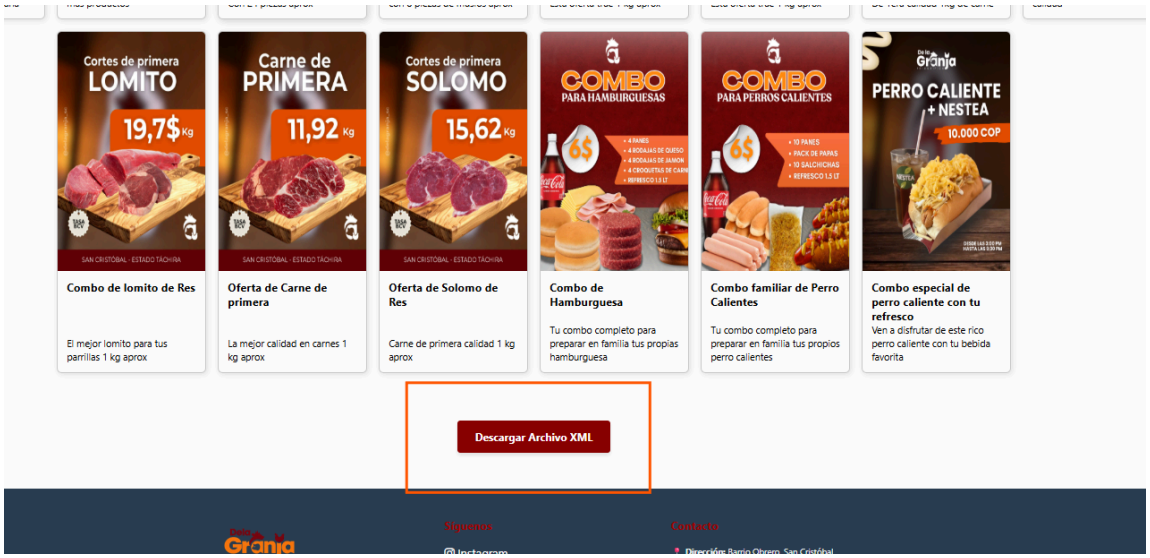
Casos de uso reales mediante el botón de descarga:

1. **Sincronización para Clientes B2B:** Si un restaurante o un cliente mayorista de San Cristóbal desea integrar nuestros precios actualizados en su propio sistema de gestión,

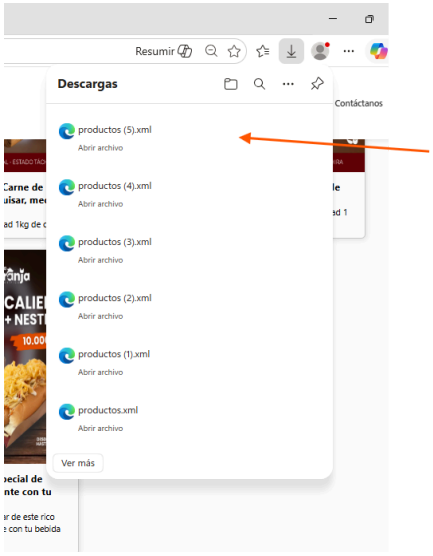
puede descargar el XML directamente desde nuestra web para procesar el pedido de forma automatizada.

2. **Portabilidad de Inventario:** El dueño del negocio (tu hermano) puede descargar el archivo en cualquier momento para tener un respaldo rápido del catálogo que está publicado, asegurándose de que los datos exportados cumplen con las reglas de negocio gracias a la validación previa del XSD.
3. **Intercambio de Información:** Al ser un formato estándar, el botón de descarga permite que la información de "De la Granja" sea compatible con cualquier otro software externo sin necesidad de copiar y pegar manualmente los datos de la web.

A continuación una captura de pantalla de la página web



Pruebas hechas:



Se abre en Visual Studio:

productos.xml

C: > Users > Roymer > Downloads > productos.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<catalogoGranja xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="catalogoGranja.xsd">

    <producto id="CMB-001" categoria="combo">
        <nombre>Combo Navideño 2026</nombre>
        <descripcion>Todo lo que necesitas para tus hallacas</descripcion>
        <detalles>
            <precio moneda="USD">57.00</precio>
            <disponibilidad>en stock</disponibilidad>
        </detalles>
    </producto>

    <producto id="CMB-002" categoria="combo">
        <nombre>Combo de la Granja 1</nombre>
        <descripcion>Es el combo pequeño y económico para tu semana</descripcion>
        <detalles>
            <precio moneda="USD">25</precio>
            <disponibilidad>en stock</disponibilidad>
        </detalles>
    </producto>
    <producto id="CMB-003" categoria="combo">
        <nombre>Combo de la Granja 2</nombre>
        <descripcion>Es el combo familiar</descripcion>
        <detalles>
            <precio moneda="USD">40.00</precio>
            <disponibilidad>en stock</disponibilidad>
        </detalles>
    </producto>
    <producto id="AVE-001" categoria="pollo">
        <nombre>Ofertas de alas de Pollo</nombre>
        <descripcion>Con 24 piezas aprox</descripcion>
        <detalles>
            <precio moneda="USD">4.78</precio>
            <cantidad>24</cantidad>
            <disponibilidad>en stock</disponibilidad>
        </detalles>
    </producto>
    <producto id="AVE-002" categoria="pollo">
        <nombre>Oferta de Muslos de Pollo</nombre>
        <descripcion>Con 8 piezas aprox</descripcion>
        <detalles>
            <precio moneda="USD">3.75</precio>
            <cantidad>8</cantidad>
            <disponibilidad>en stock</disponibilidad>
        </detalles>
    </producto>
```

Restricted Mode

Conclusión

La realización de este proyecto intermodular ha demostrado la importancia crítica que tienen los Lenguajes de Marcas en la estructuración y gestión de la información en entornos web reales. A través del caso práctico de la empresa "De la Granja", se ha logrado abstraer un catálogo visual (presente en la interfaz HTML) y convertirlo en un formato de datos procesable y estandarizado mediante XML.

El desarrollo del esquema de validación XSD ha sido el pilar fundamental para garantizar la integridad del sistema. La aplicación de restricciones avanzadas —como el uso de expresiones regulares para los identificadores, la tipificación estricta de variables monetarias y la delimitación de inventario mediante enumeraciones— asegura que el catálogo exportado esté libre de errores humanos.

En conclusión, la integración de este archivo XML descargable no solo cumple con un requisito académico, sino que dota al proyecto "De la Granja" de una herramienta profesional. Este sistema sienta las bases para futuras escalabilidades del negocio, permitiendo sincronizar precios con plataformas externas, facilitar ventas B2B y mantener un control de inventario seguro y eficiente.